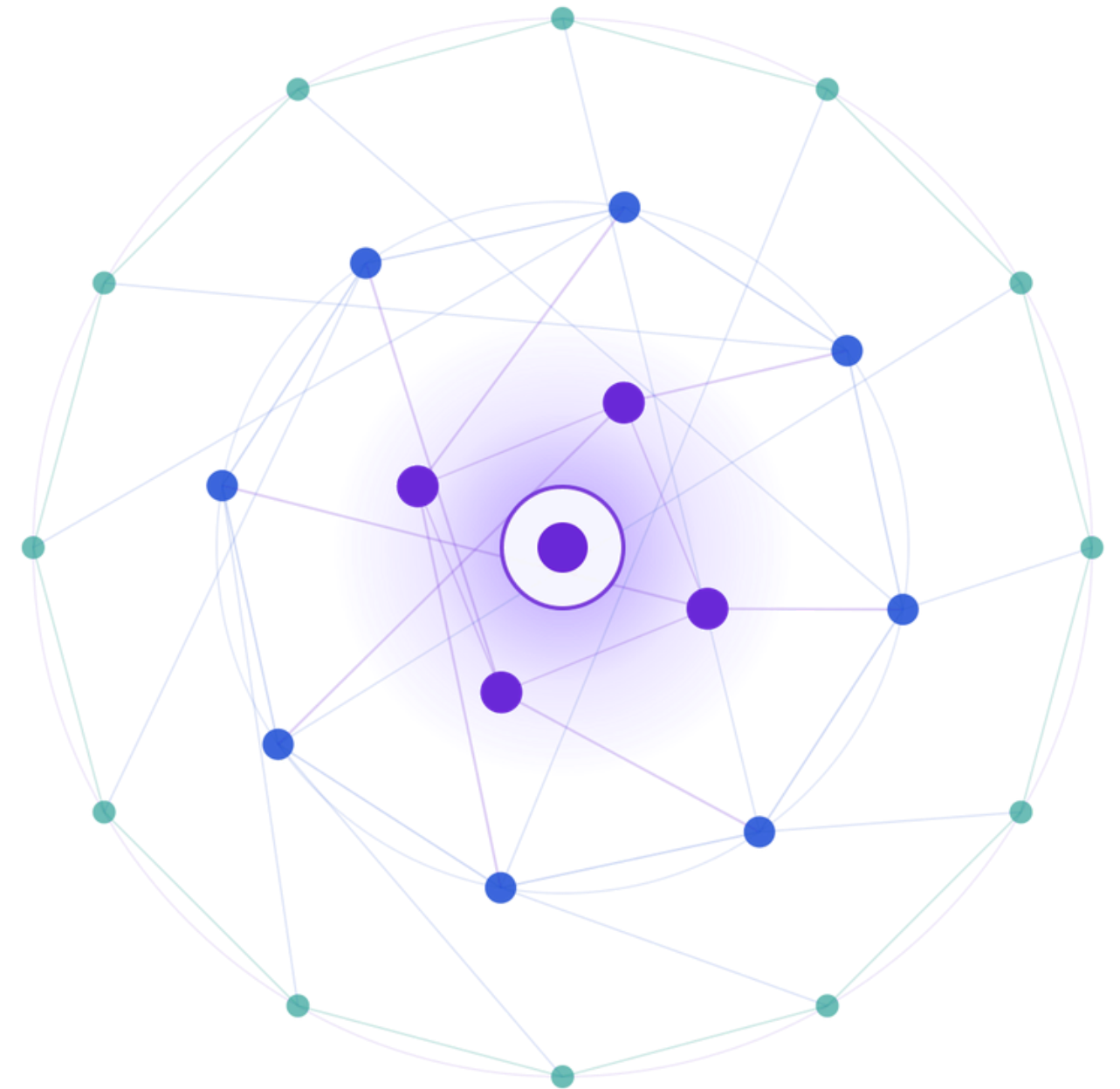


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 02
- Team Name - Tech Visionary Knights (TVK)
- College Name - SRMIST, Ktr campus
- Member Name & Email 1- Bala Tharun R, balatharunr@gmail.com
- Member Name & Email 2-Nishanth Kannan, Nishu28n18v7d@gmail.com
- Member Name & Email 3- Sandhiya S, sandhiya.official86@gmail.com
- Member Name & Email 4- Aanish Nithin A, Aanishnithin07@gmail.com
- Submission Github link - <https://github.com/sandhiya-s86/CacheCore>



Theme

- Intelligent Semantic Caching for Large Language Model Infrastructure
- AI Systems Engineering, Cost Optimization, and Low-Latency Query Execution
- LLM API queries are notoriously slow, compute-heavy, and expensive, especially when handling repetitive or semantically equivalent user requests.
- Decouple redundant model inference from runtime costs by intercepting queries at the semantic layer, delivering near-zero latency and drastic operational savings.

Existing Solutions & Gaps

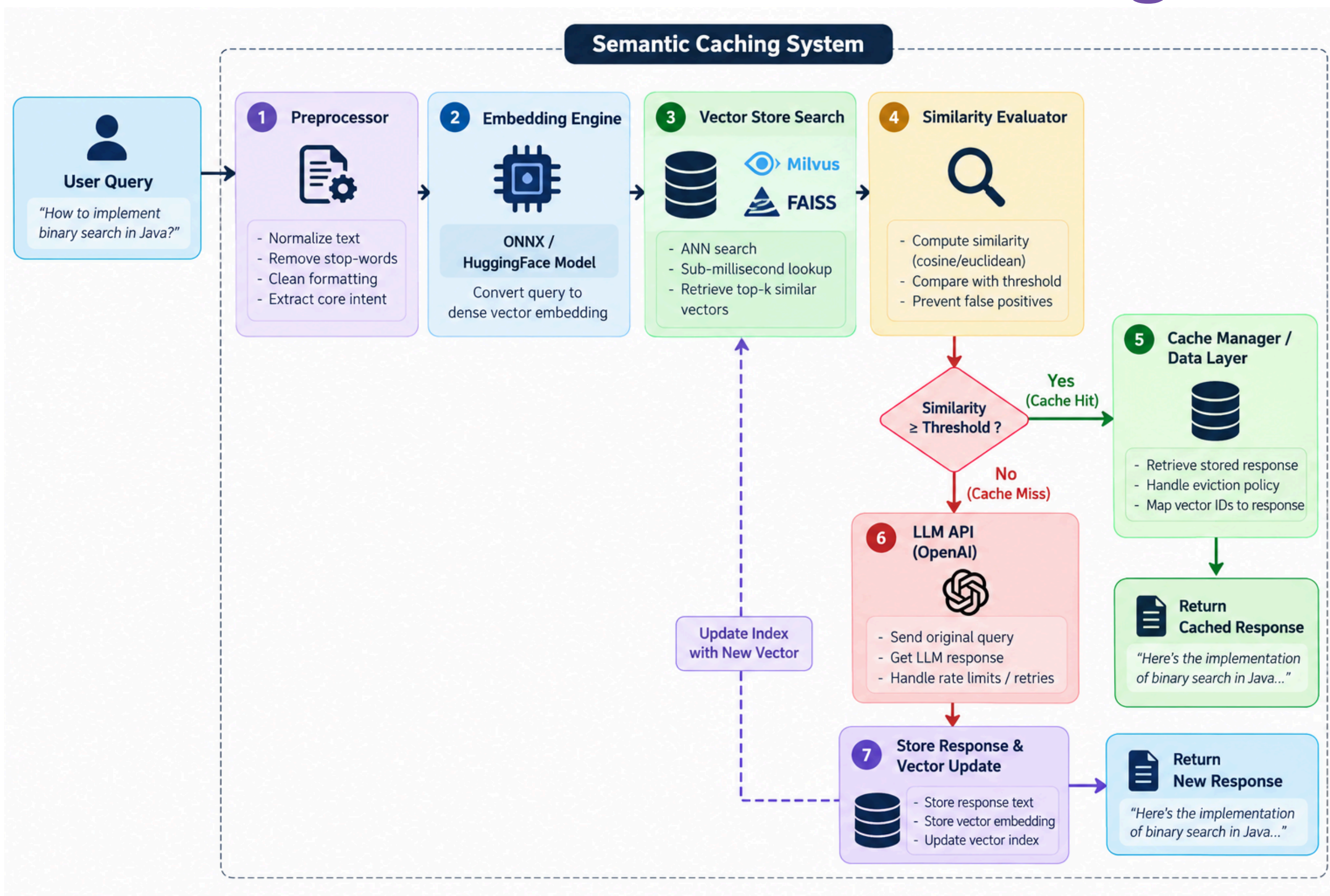
Current Approaches:

- Exact String Matching (Redis, Memcached): Standard key-value caches match raw text byte-for-byte.
- Direct LLM Invocations: Every incoming prompt hits cloud APIs (OpenAI, Anthropic) directly.

Critical Gaps:

- Zero Semantic Understanding: "What is the capital of France?" and "Tell me France's capital city" generate two separate full-price API round-trips.
- Latency Bottlenecks: Standard API round-trips average 800ms–2500ms, throttling real-time UX.
- Linear Cost Scaling: Costs scale linearly with request volume, penalizing high-concurrency applications.
- Token Waste: Repetitive context windows consume precious API rate limits and token quotas.

Our Solutions & Architecture Diagram



Demo & Product Walkthrough

- Step 1: Cold Query Execution (Cache Miss)
 - Prompt: "How does semantic caching work?"
 - Status: Cache Miss (0 vectors matched above 0.85 threshold).
 - System route: Routes to LLM backend, captures completion, inserts prompt vector into cache.
 - Response Time: ~1420ms | Cost: \$0.003
- Step 2: Semantically Equivalent Query (Cache Hit)
 - Prompt: "Explain the mechanism behind semantic cache."
 - Status: Cache Hit (Cosine similarity score: 0.93 against stored vector).
 - System route: Directly returned from local store without calling remote API.
 - Response Time: ~18ms | Cost: \$0.000
- Step 3: Admin Telemetry & Metrics
 - Live monitoring of Hit-to-Miss ratio, API spend saved, and latency percentiles (p50, p95, p99).

Tools and tech stack used

- Core Framework: GPTCache, Python 3.11
- Vector Database: Milvus / FAISS (Vector indexing and nearest-neighbor search)
- Embedding Models: HuggingFace all-MiniLM-L6-v2 / OpenAI text-embedding-3-small / ONNX Runtime
- Scalar Storage / Cache: Redis / SQLite (completion storage and metadata mapping)
- LLM Engine: LangChain / OpenAI API (fallback completion provider)
- Benchmarking & Telemetry: Prometheus, Locust (load testing), FastAPI (middleware wrapper)

Impact & Use case

Key Use Cases:

- Enterprise Customer Support Bots: Repetitive tier-1 FAQs resolved instantly without consuming quota.
- RAG / Search Engines: Common knowledge retrieval queries bypass expensive rerankers and generative steps.
- Autonomous AI Agents: Reduces redundant loop queries during iterative task execution.
- High-Traffic EdTech Platforms: Instant answers for students querying common homework and syllabus topics.

Measurable Impact:

- Cost Reduction: Up to 60–80% reduction in LLM inference bills on repeated query distributions.
- Latency Improvement: 10x–50x speedup for matched queries (from ~1.5s down to sub-30ms).
- Rate-Limit Resilience: Prevents hitting external API TPM/RPM (Tokens/Requests Per Minute) limits.

Innovation highlights, results and limitations

- Key Innovations:
 - Configurable Similarity Thresholds: Dynamic cosine boundaries to balance precision vs. recall.
 - Decoupled Architecture: Pluggable embeddings, vector DBs, and LLM backends via clean abstraction layers.
 - Local Embedding Inference: Embeddings generated on local CPU/GPU to ensure minimum overhead.
- Results:
 - Hit Latency: < 25ms average response time on hit.
 - Cache Precision: > 96% accuracy on semantic equivalence at a 0.88 cosine cutoff.
- Technical Limitations:
 - Stale Information: Cached answers can become outdated if real-time ground truth changes.
 - Cold Start Phase: Zero initial performance gains until the cache warms up with baseline data.
 - Context-Sensitive Prompts: Highly personalized or session-dependent queries can lead to context bleed if not properly keyed by user ID.

What's next

- Dynamic TTL & Eviction Policies: Context-aware cache invalidation based on topic volatility (e.g., news expires faster than math definitions).
- Multi-Modal Caching: Extend similarity matching to image, audio, and structured code inputs.
- Federated / Distributed Cache Nodes: Multi-region edge caching to bring similarity search geographically closer to end users.
- Hybrid Search Evaluator: Combine lexical BM25 filtering with vector similarity to eradicate edge-case false positives.

Brownie points slide(differentiation)

Beyond Naive Caching:

- Session-Scoped Namespace Isolation: Prevents multi-tenant data leaks by partitioning vector namespaces per tenant/session.
- Asynchronous Vector Upsertion: Updates to the vector store happen non-blocking on a background thread pool, eliminating write latency on cache misses.
- Model-Agnostic Output Adaptation: Cached responses can be reformatted or injected with dynamic variables (e.g., user name, timestamps) before rendering.
- Cost & Carbon Dashboard: Real-time visual tracking quantifying dollars saved and computational emissions offset per transaction.

Checklist- Updated on Public GitHub

Public Repository URL: <https://github.com/sandhiya-s86/CacheCore>

- [x] Source Code: Fully functional middleware pipeline with clean modular packaging.
- [x] README.md: Clear architecture diagrams, installation instructions, and quick-start scripts.
- [x] Live Benchmarks: Automated benchmark scripts comparing raw LLM calls vs. cache hits.
- [x] Docker Compose: One-command setup spinning up Redis, Milvus, and the caching proxy.
- [x] Environment Configuration: Comprehensive .env.example with zero hard-coded credentials.
- [x] Open-Source Compliance: Permissive licensing (MIT / Apache-2.0) with clean dependency tracking.

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India